

Discussion 2

Jett Rosen - Sections 13 and 14

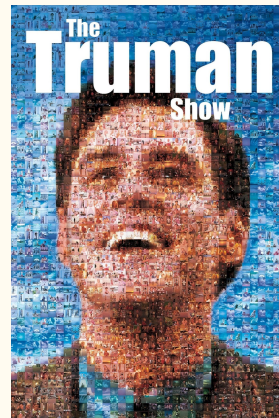


yall think the proof
in here ?

All feelings are temporary, you are loved,
you are enough, and you matter.

About Me

- I'm Jett Rosen
- 2nd semester SUGS CS student
- Second time GSI for 376 (+3 semester IA)
- Office Hours:
 - Subject to change, but currently:
 - Tuesday: 12:30-3pm LCSIB 2421
 - Thursday: 12:30-3:30pm LCSIB 2421
- Talk to me about: Machine learning, research, movies, hiking, art, Knicks basketball, One Piece, my dogs



Logistics

- Attendance QR code will be on the last slide, make sure to fill it out!
- Homework 2 is out
 - Start early!
- Slides available at (course drive)/Discussions/Slides/Jett

Exit Ticket Reflection

- Some nerves about this class being 203++
- Some excitement about learning CS theory and new problem solving methods
- I yapped too much (sorry)
- “algorithm”

On Asking Questions

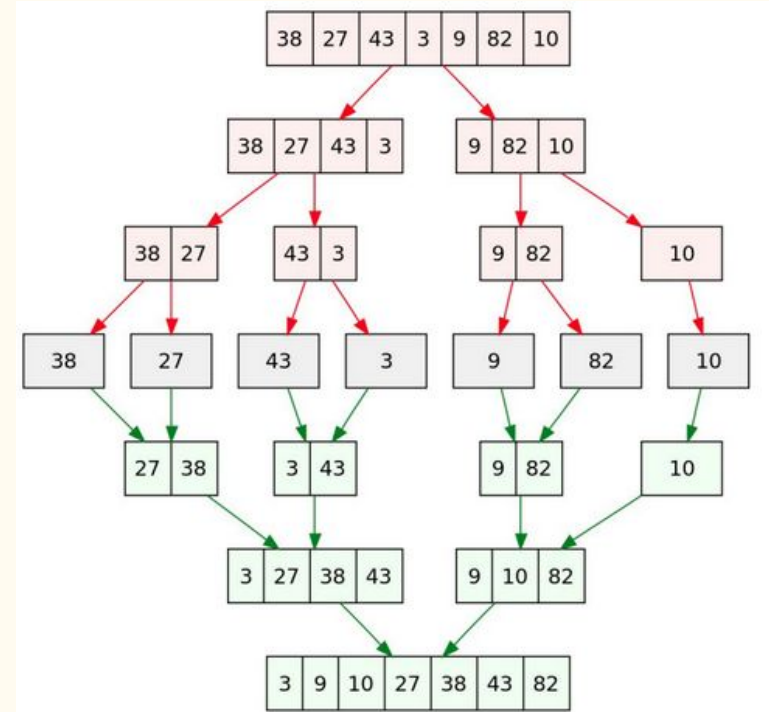
- **There are no stupid questions in EECS 376**
- It is our job to answer your questions, and it is a job we enjoy
- If you want to talk about **anything** relating to the class or CS in general, we're happy to help!

Divide and Conquer

1. Divide problem into smaller subproblems
2. Solve those problems recursively
3. Combine the smaller solutions to get a bigger solution

Proof structure:

- Base case → when to stop dividing
- Inductive step → assuming your recursive calls are correct, explain why your combine step is correct



Divide and Conquer is Inductive by Nature

Prove by Induction	Correctness Proof for D&C
Prove that $P(0)$ is true	Prove that the base case(s) is/are correct
Assuming $P(k)$ is true, prove $P(k + 1)$ is true	Assuming the outputs of the recursive calls are correct, prove that the final output (for the original input) is also correct

Divide and Conquer Runtime

- Recursive algorithms are weird: My best advice is to **never unroll the recursion**
- Consider an arbitrary divide-and-conquer algorithm that breaks a problem of size n into:
 - k smaller subproblems where $k \geq 1$
 - Each subproblem is of size n/b , where $b > 1$
 - The cost of splitting and combining results is $O(n^d)$ where $d \geq 0$
- This algorithm has the following recurrence:

$$T(n) = kT(n/b) + O(n^d)$$

The Master Theorem

Theorem 1 (The Master Theorem). *Let $T(n) = k \cdot T(n/b) + O(n^d)$ for constants $k \geq 1, b > 1, d \geq 0$. Then*

$$T(n) = \begin{cases} O(n^d) & \text{if } k < b^d \\ O(n^d \log n) & \text{if } k = b^d \\ O(n^{\log_b k}) & \text{if } k > b^d. \end{cases}$$

Moreover, if the additive term of the recurrence is a big- Θ expression (rather than a big- O one), then all the closed-form solutions for $T(n)$ are big- Θ bounds as well.

The Master Theorem also generalizes to cases where the additive term (representing the “combine” runtime) has logarithmic factors.

Theorem 2 (The Master Theorem with Log Factors). *Let $T(n) = k \cdot T(n/b) + \Theta(n^d \log^w n)$ for constants $k \geq 1, b > 1, d \geq 0, w \geq 0$. Then*

$$T(n) = \begin{cases} \Theta(n^d \log^w n) & \text{if } k < b^d \\ \Theta(n^d \log^{w+1} n) & \text{if } k = b^d \\ \Theta(n^{\log_b k}) & \text{if } k > b^d. \end{cases}$$

This is a generalization of the Master Theorem in that if $w = 0$, then we obtain the ordinary Master Theorem ([Theorem 1](#)).

Aside: Give an Algorithm...

- You are going to give *a lot* of algorithms this semester.
- This means:
 - **High-level English description** of the algorithm's structure, main steps, and what they do
 - **Precise pseudocode** for the algorithm and any subroutines (helper functions)
 - **Correctness analysis:** Prove that your algorithm outputs a correct answer on any (valid) input
 - Often start by showing some "key property" of the problem that your algorithm exploits
 - **Runtime analysis:** Give an asymptotic bound on the (worst-case) running time as a function of input size (A Big-O bound will suffice unless otherwise mentioned)

Exit Ticket

